



Wyższa Szkoła Ekonomii i Informatyki w Krakowie

Laboratoria, 07.03.2026r.

Architektura Aplikacji w Pythonie [Lab 1] wprowadzenie, zaawansowane konstrukcje języka

Python

Michał Madejski

Kraków, 2026

Agenda



1. **Wprowadzenie** (Kontakt, Cele przedmiotu, Literatura, Forma prowadzenia zajęć, Zaliczenie)
2. **Dekoratory** (Definicja, Przypomnienie specyfiki funkcji w Python, Składnia dekoratorów, Łącuchowanie dekoratorów, Dekoratory z argumentem, Dekorator klasy, Przykłady często stosowanych dekoratorów, zadania)
3. **Deskryptory** (Definicja, Projektowanie deskryptorów, Metody protokołu deskryptora, Metoda specjalna, Klasyfikacja deskryptorów, zadania)
4. **Generatory i iteratory** (Definicja, Generatory poprzez wyrażenia, yield, next, itertools, Metody magiczne `__next__` i `__iter__`, close, send, throw, zadania)

Kontakt



mmadejski@wsei.edu.pl



Microsoft Teams

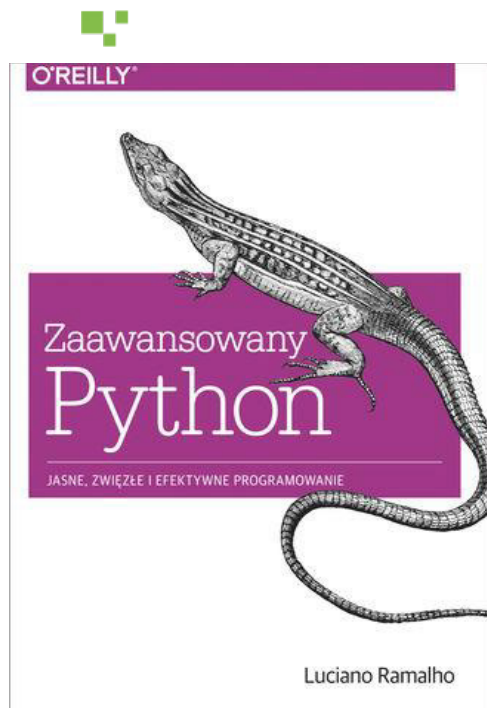
W celu umówienia konsultacji proszę
o kontakt indywidualny.

Cele przedmiotu

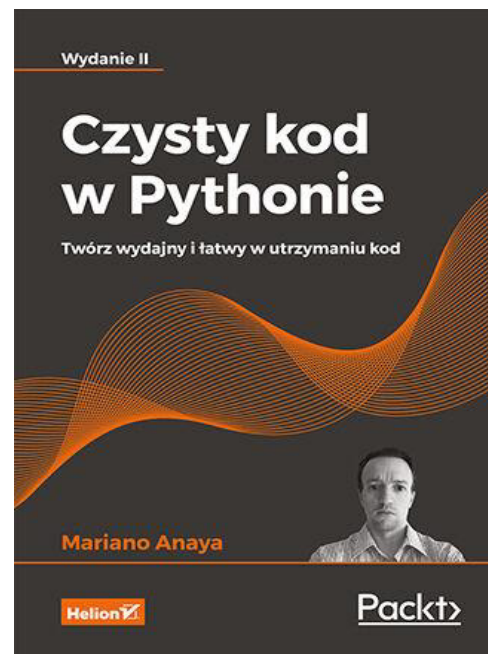


1. Zapoznanie z zaawansowaną konstrukcją języka Python.
2. Opanowanie umiejętności implementowania wielowątkowości.
3. Poznanie zagadnień związanych z integracją z bazami danych oraz zewnętrznymi serwisami i testowaniem programów.
4. Implementacja wzorców projektowych w Pythonie.
5. Wprowadzenie do zagadnień związanych z rozwojem aplikacji.

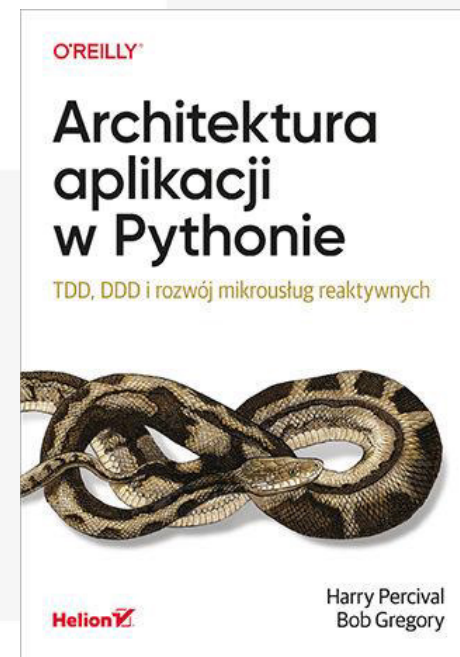
Literatura przedmiotu



Ramalho Luciano, „Zaawansowany Python. Jasne, zwięzłe i efektywne programowanie”, Wydawnictwo APN Promise, Warszawa 2015



Anaya Mariano, „Czysty kod w Pythonie. Twórz wydajny i łatwy w utrzymaniu kod”, Wydawnictwo Helion, Gliwice 2022

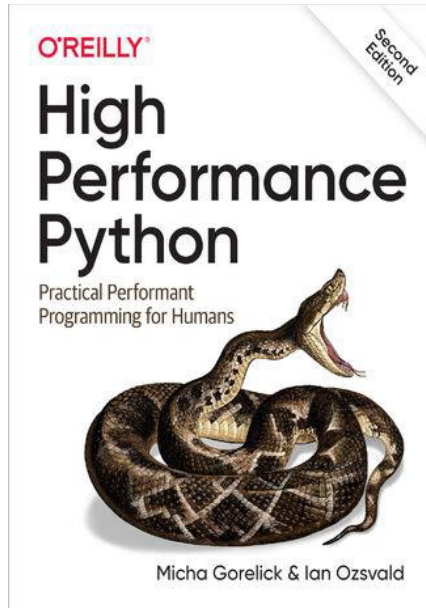


Percival Harry, Gregory Bob, „Architektura aplikacji w Pythonie. TDD, DDD i rozwój mikro usług reaktywnych”, Wydawnictwo Helion, Gliwice 2020

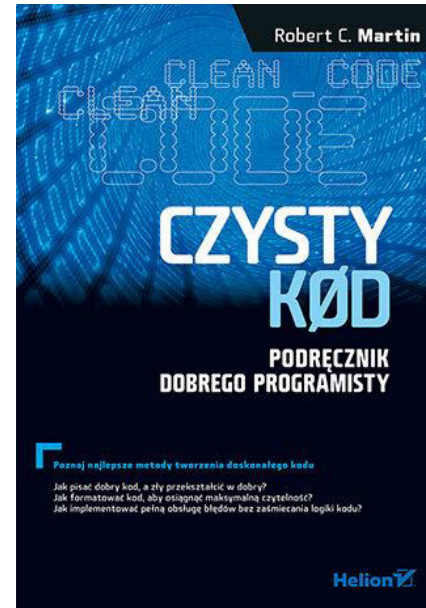


Sweigart Albert, „Programowanie w Pythonie dla średnio zaawansowanych. Najlepsze praktyki tworzenia czystego kodu”, Wydawnictwo Helion, Gliwice 2021

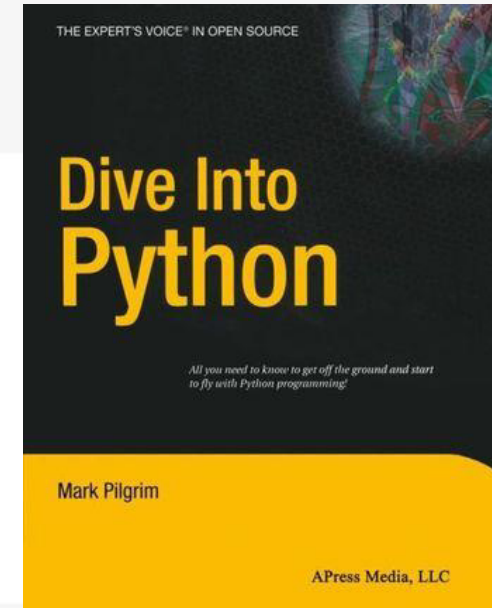
Literatura dodatkowa



Micha Gorelick, Ian Ozsvald, "High Performance Python, 2nd Edition", O'Reilly Media, Inc. 2020

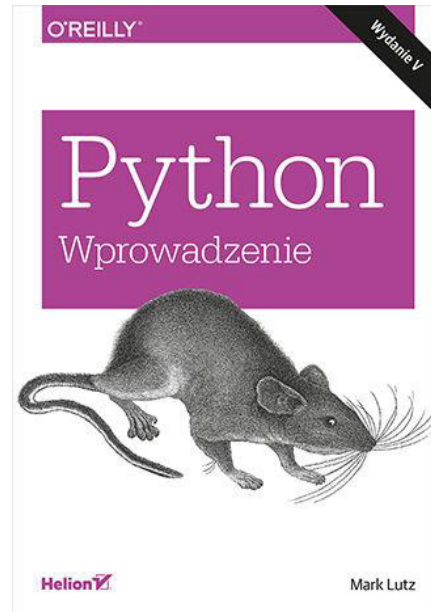


Martin Robert C., „Czysty kod. Podręcznik dobrego programisty”, Wydawnictwo Helion, Gliwice 201

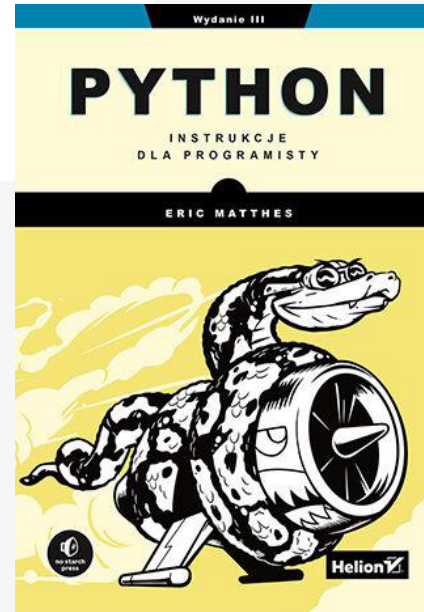


Pilgrim Mark, „Dive into Python”,
podręcznik online:
<https://diveintopython3.problemsolving.io/>

Literatura uzupełniająca [podstawy języka Python]

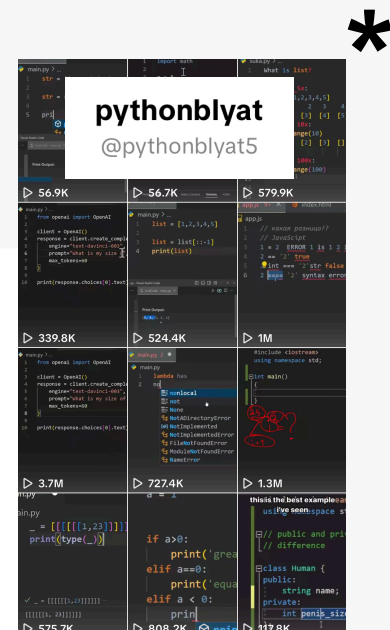
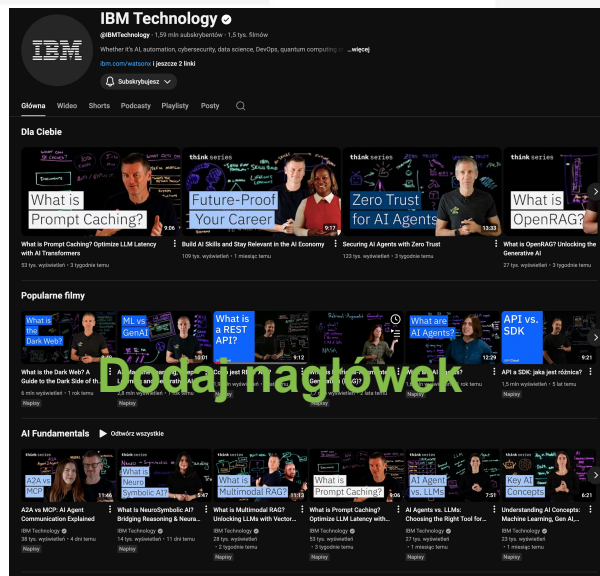
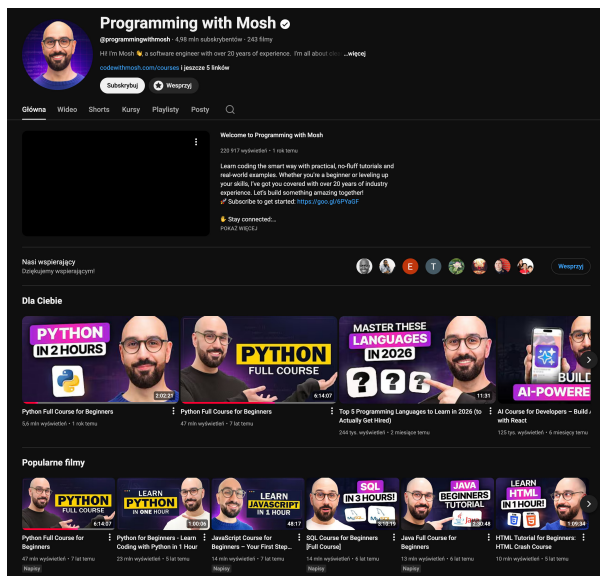


Mark Lutz, "Python. Wprowadzenie. Wydanie V", Wydawnictwo Helion, Gliwice 2020



Eric Matthes "Python. Instrukcje dla programisty. Wydanie III", Wydawnictwo Helion, Gliwice 2023

Polecane kanały



Forma prowadzenia zajęć



Forma prowadzenia zajęć:

1. Przedstawienie konceptów i dobrych praktyk,
2. Demonstracja implementacji,
3. Indywidualne rozwiązywanie zadań programistycznych z pomocą prowadzącego.

Zaliczenie

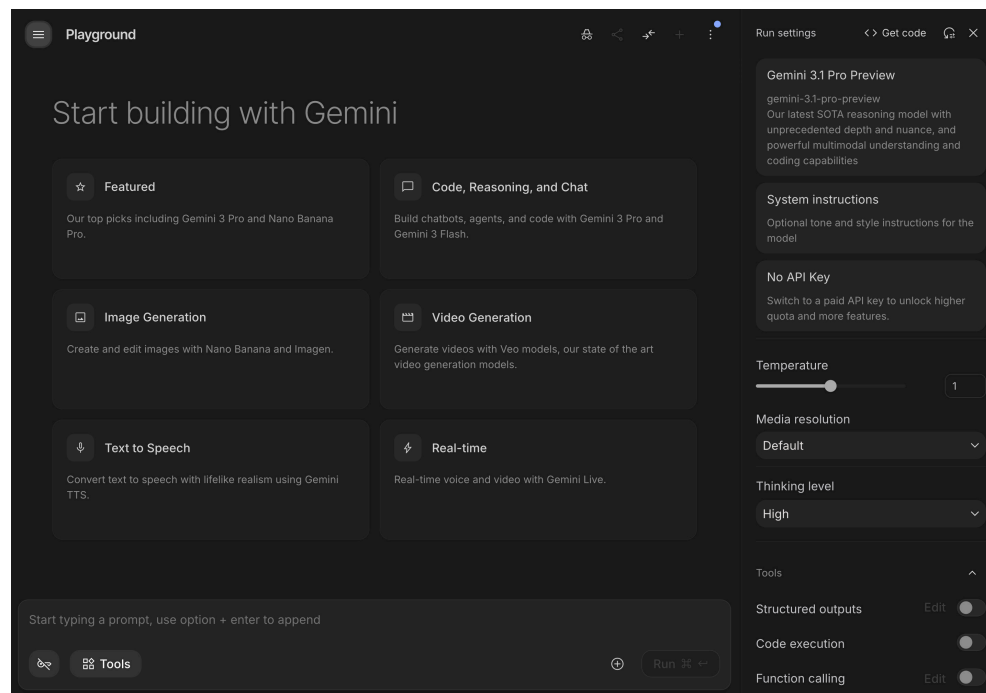


Forma zaliczenia:

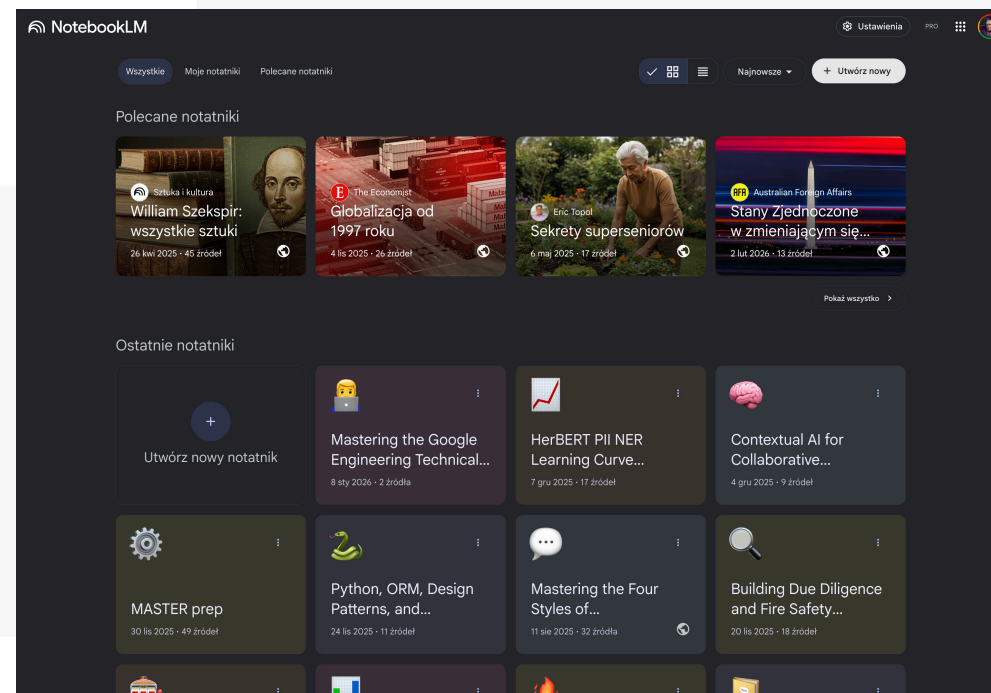
Aby zaliczyć efekt przedmiotowy odnoszący się do sprawdzenia umiejętności, który realizuje określony efekt kierunkowy, student/k w trakcie semestru jest zobowiązany do **wykonania zadań programistycznych** zleconych przez prowadzącego zajęcia oraz projektu. Każde zadanie oraz projekt będzie miało przypisaną określoną liczbę punktów, ich sumaryczna liczba punktów będzie warunkowała uzyskanie ocen.

Po Laboratorium 5 zostanie przekazany **zestaw zaliczeniowy**, składający się z krótkich zadań programistycznych oraz jednego zadania "projektowego".

Wykorzystanie narzędzi AI



Google AI Studio



NotebookLM

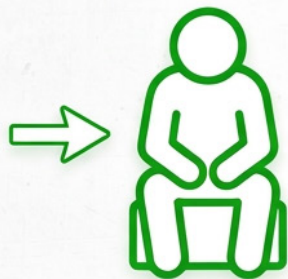
Tutor prompt <https://bit.ly/tutor-prompt>



JESTEM TWOIM TUTOREM, NIE WYKONAWCĄ.
POMAGAM CI ROZUMIEĆ, MYŚLEĆ I ROZWIĄZYWAĆ SAMODZIELNIE.
NIE PISZĘ KODU ZA CIEBIE.



GOTOWIEC



**METODA TUTORINGU
PROGRAMOWANIA
W PYTHONIE**



**METODA TUTORINGU PROGRAMOWANIA W PYTHONIE
W PYTHONIE**

ZASADY PRACY:
1. DIAGNOZA

**2. WSKAZÓWKI
KONCEPCYJNE**

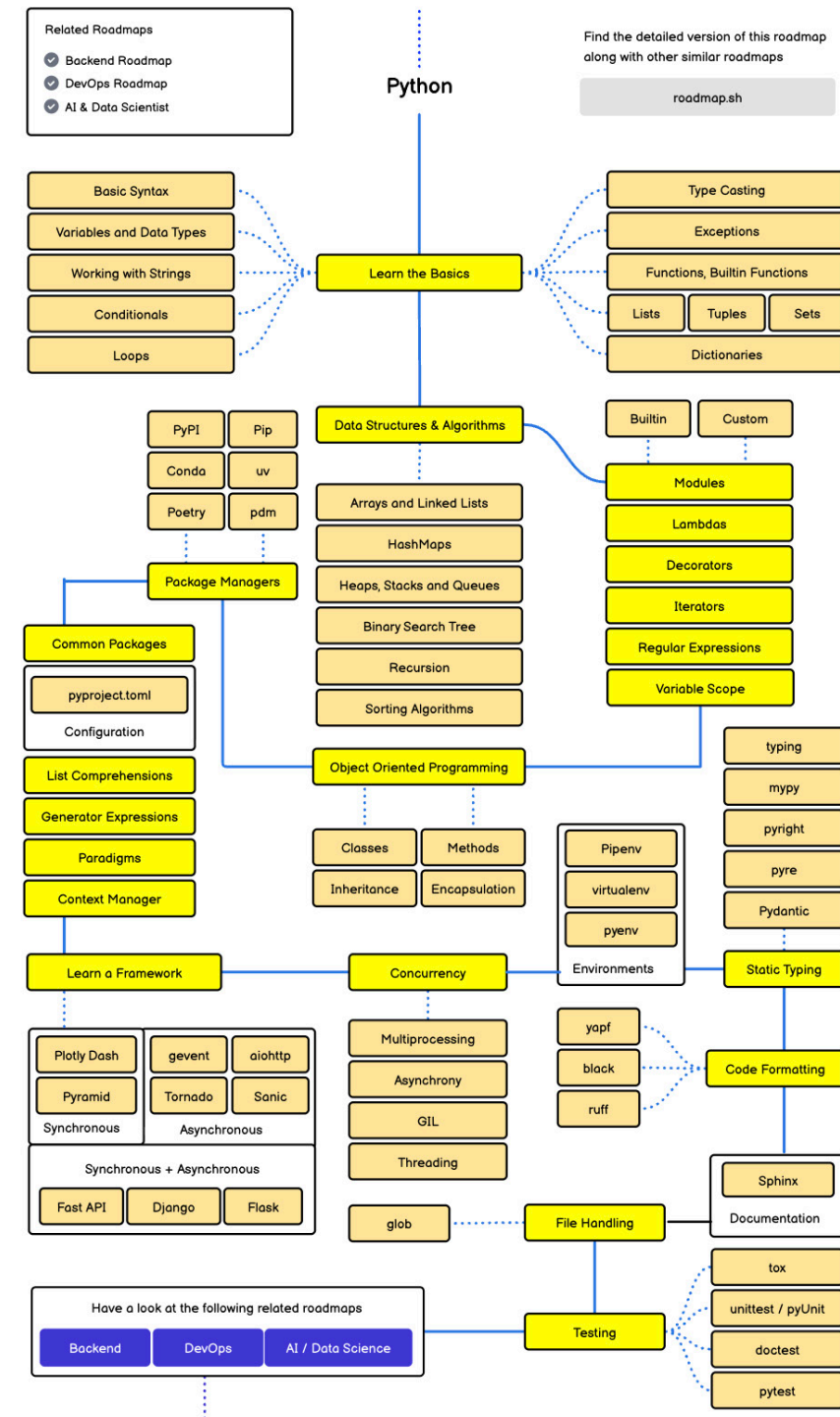
**3. PLAN
ROZWIĄZANIA**

4. PSEUDOKOD

**5. CZĘŚCIOWY
KOD**

**6. PEŁNY KOD
(OSTATECZNOŚĆ)**

- > <https://roadmap.sh/python> 5min
- > <https://bit.ly/project-match> copy
- > <https://roadmap.sh/ai/chat> fill your info, paste



Dekoratory



W Pythonie **dekorator** to wzorzec projektowy, który pozwala zmodyfikować funkcjonalność funkcji poprzez jej opakowanie (*wrapping*) w inną funkcję. Dekoratory zostały zaimplementowane w Python 2.4a2 wg standardu [PEP-318](#).

Praktyczne zastosowania dekoratorów:

- Logowanie
- Profilowanie kodu i monitorowanie czasów wykonania
- Walidacja danych
- Obsługa błędów
- Cache'owanie
- Kontrola dostępu, autentykacja i autoryzacja (używane w Django i Flask)
- Transformacja danych

Dobra praktyka:
Zaleca się tworzenie dekoratorów, które są proste i odpowiedzialne za jedną operację.

PEP <https://peps.python.org/pep-0008/>



Introduction

This document gives coding conventions for the Python code comprising the standard library in the main Python distribution. Please see the companion informational PEP describing style guidelines for the C code in the C implementation of Python.

This document and PEP 257 (Docstring Conventions) were adapted from Guido's original Python Style Guide essay, with some additions from Barry's style guide [2].

This style guide evolves over time as additional conventions are identified and past conventions are rendered obsolete by changes in the language itself.

Many projects have their own coding style guidelines. In the event of any conflicts, such project-specific guides take precedence for that project.

VSC / Cursor / Antigravity **AutoPep8** extension

<https://marketplace.visualstudio.com/items?itemName=ms-python.autopep8>



Przypomnienie specyfiki funkcji w Python



- **Wszystko w Pythonie jest obiektem** – nawet funkcja. Funkcje w Pythonie mogą być przypisywane do zmiennych, przekazywane jako argumenty do innych funkcji oraz zwracane jako wartości z innych funkcji.

```
1  # Wszystko w Pythonie jest obiektem
2
3  def jaki_to_jezyk(jezyk = "Python"):
4      return "Jest to " + jezyk
5
6  # Możemy przypisywać funkcje do zmiennej
7  zmienna = jaki_to_jezyk
8  print(zmienna())
9  # Wynik: "Jest to Python"
```

- W Pythonie możemy **zagnieżdżać (nest) funkcje** - definiować funkcje wewnątrz innej funkcji

```
13  #Możemy zagnieżdżać funkcje
14  def wsei():
15      print("jestesmy wewnatrz funkcji wsei()")
16
17      def python():
18          return "jestesmy wewnatrz funkcji python()"
19
20      print(python())
21      print("jestesmy znowu wewnatrz funkcji wsei()")
22
23  print(wsei())
24  #Wynik: jestesmy wewnatrz funkcji wsei()
25  #      jestesmy wewnatrz funkcji python()
26  #      jestesmy znowu wewnatrz funkcji wsei()
```

Przypomnienie specyfiki funkcji w Python



- Możemy **zwracać funkcje** wewnątrz innej funkcji

```
29 #Możemy zwracać funkcje będąc wewnątrz innej funkcji
30 def wsei():
31     def python():
32         return "jesteśmy wewnątrz funkcji python()"
33
34     return python
35
36 print(wsei())
37 #Wynik: <function wsei.<locals>.python at 0x0000017B752098B0>
```

- Możemy przekazywać **funkcję jako argument** innej funkcji

```
40 #Możemy przekazywać funkcje jako argument innej funkcji
41 def python():
42     return "To jest python!"
43
44 def jaki_to_jezyk(func):
45     print("Jaki to język?")
46     print(func())
47
48 jaki_to_jezyk(python)
49 #Wynik: Jaki to język?
50 #      To jest python!
```

Składnia dekoratorów



W dekoratorach, funkcje są przekazywane jako argument do innej funkcji, a następnie wywoływane wewnątrz funkcji opakowującej.

Składnia:

```
53  @dekorator
54  def funkcja():
55      # Ciało funkcji
```

Składnia dekoratorów



Funkcja dekorująca (*modifier*) - Jest to funkcja, która przyjmuje inną funkcję jako argument i zwraca nową funkcję lub modyfikuje zachowanie funkcji, która została przekazana jako argument. **Funkcja dekorowana** (*original*) - Jest to funkcja, której zachowanie jest modyfikowane przez funkcję dekorującą. Funkcja dekorowana jest przekazywana jako argument do funkcji dekorującej.

Przykład:

```
def dekorator_dodajacy_logi(oryginalna_funkcja):  
    def opakowana_funkcja():  
        print("Wykonanie przed funkcją")  
        oryginalna_funkcja()  
        print("Wykonanie po funkcji")  
    return opakowana_funkcja  
  
@dekorator_dodajacy_logi  
def wypisz_komunikat():  
    print("To jest funkcja do udekorowania")  
  
wypisz_komunikat()  
  
# Rezultat:  
# Wykonanie przed funkcją  
# To jest funkcja do udekorowania  
# Wykonanie po funkcji
```


Składnia dekoratorów



Definiowanie dekoratorów z ***args** i ****kwargs** umożliwia tworzenie bardziej ogólnych dekoratorów, które mogą być stosowane do funkcji o różnych różnych ilościach argumentów. Parametr ***args** pozwala dekoratorowi przyjąć dowolną liczbę pozycyjnych argumentów. Parametr ****kwargs** umożliwia dekoratorowi przyjęcie dowolnej liczby argumentów nazwanych.

```
# *args -> zbiera argumenty pozycyjne do krotki
# **kwargs -> zbiera argumenty nazwane do słownika
def pokaz_wszystko(*args, **kwargs):
    print("Argumenty pozycyjne:", args)
    print("Argumenty nazwane:", kwargs)

pokaz_wszystko(1, 2, 3, imie="Marek", miasto="Kraków")

# Wynik:
# Argumenty pozycyjne: (1, 2, 3)
# Argumenty nazwane: {'imie': 'Marek', 'miasto': 'Kraków'}

# Nazwy "args" i "kwargs" są tylko konwencją
# Można użyć innych nazw, jeśli nadal zachowamy * i **
def pokaz_dane(*rzeczy, **ustawienia):
    print("Rzeczy:", rzeczy)
    print("Ustawienia:", ustawienia)

pokaz_dane("Python", "Git", poziom="junior", aktywny=True)

# Wynik:
# Rzeczy: ('Python', 'Git')
# Ustawienia: {'poziom': 'junior', 'aktywny': True}
```

```
# *extras zbiera dodatkowe argumenty pozycyjne do krotki
# **options zbiera argumenty nazwane do słownika
def show_order(product, *extras, **options):
    print(f"Product: {product}")

    print("Extras:")
    for extra in extras:
        print(f"- {extra}")

    print("Options:")
    for key, value in options.items():
        print(f"- {key}: {value}")

show_order("burger", "cheese", "bacon", spicy=True, takeaway=False)

# Wynik:
# Product: burger
# Extras:
# - cheese
# - bacon
# Options:
# - spicy: True
# - takeaway: False
```

```
71 #przykład definicji dekoratora z *args i **kwargs
72 def sumowanie_dekorator(func):
73     def wewnetrzna_funkcja(*args, **kwargs):
74         print("Argumenty przekazane do funkcji:", args, kwargs)
75         wynik = func(*args, **kwargs)
76         print("Wynik funkcji:", wynik)
77         return wynik
78     return wewnetrzna_funkcja
79
80 @sumowanie_dekorator
81 def sumowanie(a, b, c=0):
82     return a + b + c
83
84 print(sumowanie(1, 2, c=3))
85 #Wynik: Argumenty przekazane do funkcji: (1, 2) {'c': 3}
86 #      Wynik funkcji: 6
87 #      6
```

Dobra praktyka -@functools.wraps(func)



Gdy udekorujemy funkcję za pomocą dekoratora, funkcja oryginalna zostaje zastąpiona przez funkcję wewnętrzną, która jest zwracana przez dekorator.

Dla poprzedniego przykładu:

```
89     #Metadane funkcji
90     print(sumowanie.__name__)
91     #Wynik: wewnetrzna_funkcja
```

Dzięki użyciu **@functools.wraps(func)** funkcja dekorująca zachowuje metadane funkcji dekorowanej, w tym jej nazwę.

```
94     import functools
95     #Użycie functools.wraps
96     def sumowanie_dekorator(func):
97         @functools.wraps(func)
98         def wewnetrzna_funkcja(*args, **kwargs):
99             print("Argumenty przekazane do funkcji:", args, kwargs)
100             wynik = func(*args, **kwargs)
101             print("Wynik funkcji:", wynik)
102             return wynik
103         return wewnetrzna_funkcja
104
105     @sumowanie_dekorator
106     def sumowanie(a, b, c=0):
107         return a + b + c
108
109     #Metadane funkcji
110     print(sumowanie.__name__)
111     #Wynik: sumowanie
```

Łańcuchowanie dekoratorów



Łańcuchowanie dekoratorów to technika, która polega na aplikowaniu wielu dekoratorów do jednej funkcji. Można stosować jeden dekorator do funkcji, a następnie stosować kolejny dekorator do wyniku poprzedniego dekorowania.

```
117 #Łańcuchowanie dekoratorów
118 def dekorator1(func):
119     def wewnetrzna():
120         print("Jestem dekoratorem 1")
121         func()
122     return wewnetrzna
123
124 def dekorator2(func):
125     def wewnetrzna():
126         print("Jestem dekoratorem 2")
127         func()
128     return wewnetrzna
129
130 @dekorator1
131 @dekorator2
132 def funkcja():
133     print("Jestem funkcją")
134
135 funkcja()
136 #Wynik: Jestem dekoratorem 1
137 #       Jestem dekoratorem 2
138 #       Jestem funkcją
```

Dekorator z argumentem



Dekoratory z argumentem to technika, która pozwala na dostosowywanie zachowania dekoratora poprzez przekazanie mu argumentów.

```
144 # Dekorator z argumentem
145 def dekorator_z_argumentem(prefix):
146     def dekorator(func):
147         @functools.wraps(func)
148         def wewnetrzna(*args, **kwargs):
149             print(prefix, "Rozpoczynam wywołanie funkcji", func.__name__)
150             wynik = func(*args, **kwargs)
151             print(prefix, "Zakończono wywołanie funkcji", func.__name__)
152             return wynik
153         return wewnetrzna
154     return dekorator
155
156 @dekorator_z_argumentem("INFO:")
157 def funkcja(a, b):
158     return a + b
159
160 print(funkcja(2, 3))
161 #Wynik: INFO: Rozpoczynam wywołanie funkcji funkcja
162 #      INFO: Zakończono wywołanie funkcji funkcja
163 #      5
```


Dekorator klasy



Dekorator klasy może modyfikować lub rozszerzać zachowanie całej klasy i jej metod.

```
167 # Dekorator klasy
168 def dekorator_klasz(cls):
169     class WewnetrznaKlasa:
170         def __init__(self, *args, **kwargs):
171             self.wrapped = cls(*args, **kwargs)
172
173         def __getattr__(self, attr):
174             return getattr(self.wrapped, attr)
175
176         def nowa_metoda(self):
177             print("To jest nowa metoda dodana przez dekorator")
178
179     return WewnetrznaKlasa
180
181 @dekorator_klasz
182 class MojaKlasa:
183     def __init__(self, x):
184         self.x = x
185
186     def metoda(self):
187         print("To jest metoda oryginalnej klasy")
188
189 # Użycie dekorowanej klasy
190 obj = MojaKlasa(5)
191 print(obj.x)
192 print(obj.metoda())
193 print(obj.nowa_metoda())
194 #Wynik: To jest metoda oryginalnej klasy
195 #       To jest nowa metoda dodana przez dekorator
```

Dobra praktyka – 3 warunki dobrego dekoratora



1. Enkapsulacja, czyli separacja obowiązków:

Dobry dekorator powinien dodawać tylko jedną, konkretną odpowiedzialność, bez mieszania logiki biznesowej samej funkcji. Funkcja ma robić swoje główne zadanie, a dekorator ma zajmować się czymś „wokół”, np. logowaniem, mierzeniem czasu, walidacją albo kontrolą dostępu. Dzięki temu kod jest czytelniejszy i łatwiejszy w utrzymaniu

2. Ortogonalność:

Zachowanie dodawane przez dekorator powinno być możliwie niezależne od szczegółów działania dekorowanej funkcji. Innymi słowy, dekorator nie powinien być mocno przywiązany do konkretnej implementacji, tylko działać obok niej i rozszerzać ją w możliwie uniwersalny sposób. To zmniejsza sprzężenie i ułatwia stosowanie dekoratora w różnych miejscach.

3. Możliwość ponownego użycia:

Dobry dekorator powinien dać się zastosować do wielu funkcji, metod lub klas o podobnym charakterze, a nie być pisany tylko pod jeden konkretny przypadek. Jeśli działa wyłącznie dla jednej funkcji i jednego scenariusza, to często lepiej byłoby po prostu dopisać zwykłą funkcję pomocniczą. Reużywalność oznacza, że dekorator wnosi ogólny mechanizm, który można wykorzystać szerzej.

Przykłady często stosowanych dekoratorów



Lista dekoratorów: <https://wiki.python.org/moin/Decorators>
<https://github.com/lord63/awesome-python-decorator>

Kilka przykładów standardowych dekoratorów:

@property - używany do zdefiniowania metody, która zachowuje się jak atrybut klasy.

@classmethod - jest stosowany do zdefiniowania metody, która jest wywoływana na poziomie klasy, a nie na poziomie instancji.

@staticmethod - Ten dekorator jest używany do zdefiniowania metody, która jest wywoływana na poziomie klasy, ale nie ma dostępu do jej atrybutów ani metod.

DEMO



Stwórz dekorator `@timer`, który będzie mierzył i wyświetlał czas wykonania funkcji.

```
3 import time
4 import functools
5 import logging
6
7 logger = logging.getLogger(__name__)
8 def timer(func):
9     @functools.wraps(func)
10    def wrapper(*args, **kwargs):
11        start_time = time.time()
12        result = func(*args, **kwargs)
13        end_time = time.time()
14        execution_time = end_time - start_time
15        logger.warning(f"Function '{func.__name__}' execution time:
16                      {execution_time:.5f} seconds")
17        return result
18    return wrapper
19
20 @timer
21 def example_function(n):
22     total = 0
23     for i in range(n):
24         total += i
25     return total
26
27 result = example_function(1000000)
28 print("Result:", result)
```

Zadania



1. Stwórz dekorator, który będzie odpowiedzialny za wyświetlanie liczby elementów listy, jeśli jakakolwiek lista pojawi się w parametrach funkcji dekorowanej. *Protip: użyj `isinstance` do sprawdzenia czy parametr jest listą*
2. Stwórz dekorator, który będzie zapisywał w pliku *.log nazwę funkcji dekorowanej, datę oraz długość wykonania. Nazwa pliku będzie podana jako argument dekoratora. *Protip: użyj biblioteki `datetime`*

Pamiętaj o tym, żeby dekoratory przyjęły metadane funkcji dekorującej.

Deskryptory



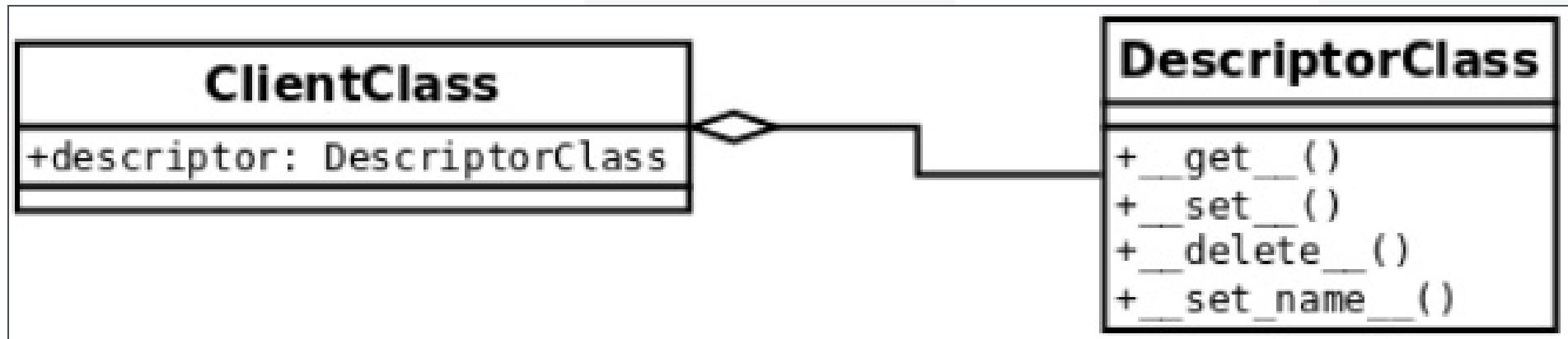
Deskryptory są mechanizmem w języku Python, który **umożliwia kontrolę dostępu do atrybutów w klasach**. Pozwalają one na zdefiniowanie niestandardowego zachowania przy dostępie do atrybutu klasy, takiego jak jego odczyt, zapis lub usunięcie.

Dobra praktyka:
Powinniśmy używać deskryptorów do szczególnych przypadków, takich jak projektowanie wewnętrznych API, bibliotek czy frameworków.

Projektowanie deskryptorów



W typowym przypadku, deskryptor jest definiowany jako osobna klasa i jest używany jako atrybut w innej klasie:



Metody protokołu deskryptora



Protokół deskryptora definiuje **metody specjalne**, które można zaimplementować w klasie deskryptora:

`__get__(self, instance, owner)` - jest wywoływana, gdy próbujemy pobrać wartość atrybutu, którego deskryptorem jest dana klasa. Parametr ***instance*** to instancja klasy, do której deskryptor jest przypisany, a ***owner*** to klasa, której instancja została stworzona.

```
1  # Prezentacja metody __get__ w deskrytorze
2  class Deskryptor:
3      def __get__(self, instance, owner):
4          print("Pobieranie wartości...")
5          return instance._wartosc
6
7  class MojaKlasa:
8      deskryptor = Deskryptor()
9
10     obj = MojaKlasa()
11     obj._wartosc = 42
12
13     print(obj.deskryptor)
14     #Wynik: Pobieranie wartości...
15     #      42
```


Metody protokołu deskryptora



`__set__(self, instance, value)` Ta metoda jest wywoływana, gdy próbujemy ustawić wartość atrybutu, którego deskryptorem jest dana klasa. Parametr ***instance*** to instancja klasy, do której deskryptor jest przypisany, a ***value*** to nowa wartość, którą próbujemy przypisać do atrybutu.

```
20 # Prezentacja metody __set__ w deskrytorze
21 class Deskryptor:
22     def __set__(self, instance, value):
23         print("Ustawianie wartości...")
24         instance._wartosc = value
25
26 class MojaKlasa:
27     deskryptor = Deskryptor()
28
29 obj = MojaKlasa()
30 obj.deskryptor = 42 # Wywołuje metodę __set__
31 #Wynik: Ustawianie wartości...
```

Metody protokołu deskryptora



`__delete__(self, instance)` -ta metoda jest wywoływana, gdy próbujemy usunąć wartość atrybut, którego deskryptorem jest dana klasa. Parametr *instance* to instancja klasy, do której deskryptor jest przypisany.

```
34 # Prezentacja metody __delete__ w deskrytorze
35 class Deskryptor:
36     def __delete__(self, instance):
37         print("Usuwanie wartości...")
38         del instance._wartosc
39
40 class MojaKlasa:
41     deskryptor = Deskryptor()
42
43 obj = MojaKlasa()
44 obj._wartosc = 42
45
46 del obj.deskryptor
47 #Wynik: Usuwanie wartości...
```

Metody protokołu deskryptora – metoda specjalna

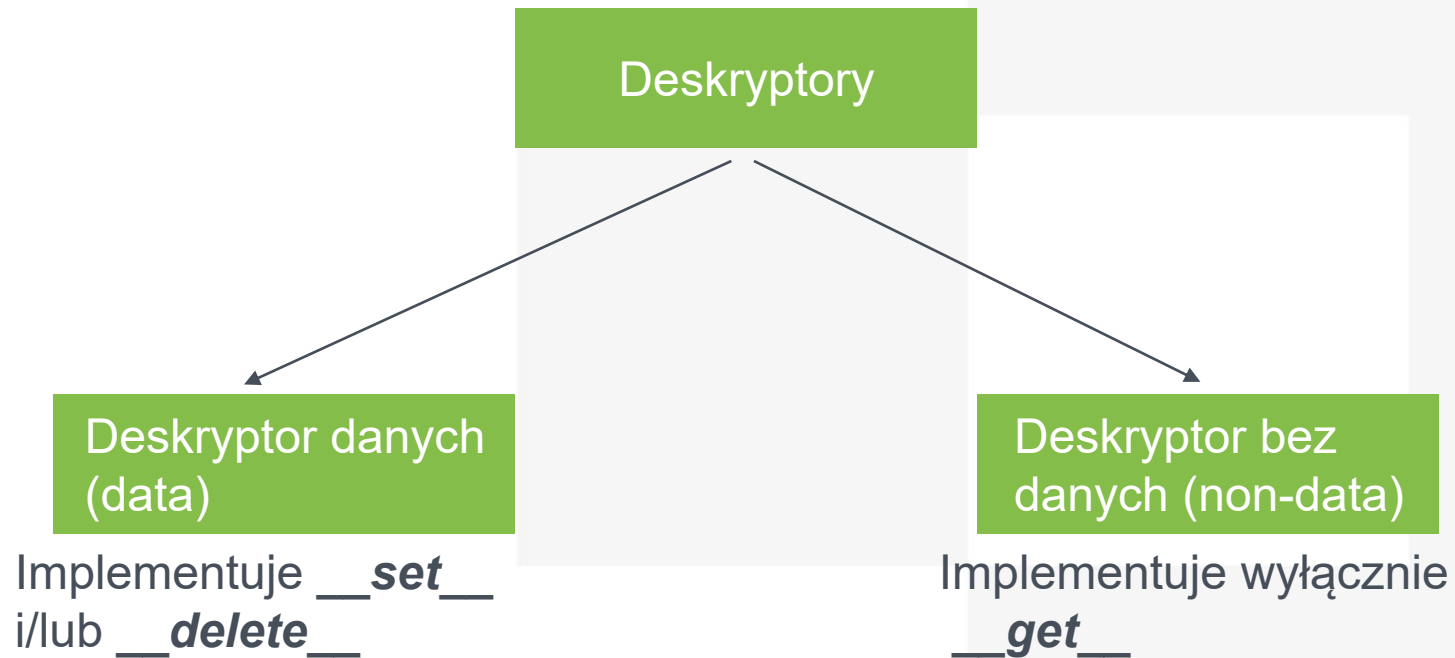


`__set_name__(self, owner, name)` - jest to metoda specjalna wprowadzona w **Pythonie 3.6**, która jest wywoływana podczas definiowania klasy i umożliwia ustawienie nazwy atrybutu w klasie, do którego deskryptor jest przypisany.

Parametr **owner** reprezentuje klasę, do której deskryptor jest przypisany, a **name** reprezentuje nazwę atrybutu.

```
49 #Prezentacja metody __set_name__ w deskrytorze
50 class BinarnyDeskryptor:
51     def __set_name__(self, owner, name):
52         self.nazwa = name
53         self.nazwa_binarna = f"{name}_binarny"
54
55     def __get__(self, instance, owner):
56         if instance is None:
57             return self
58         print(f"Pobieranie wartości atrybutu {self.nazwa}")
59         return instance.__dict__.get(self.nazwa)
60
61     def __set__(self, instance, value):
62         print(f"Ustawianie wartości atrybutu {self.nazwa} na {value}")
63         instance.__dict__[self.nazwa] = value
64         instance.__dict__[self.nazwa_binarna] = bin(value)
65
66 class MojaKlasa:
67     atrybut = BinarnyDeskryptor()
68
69 obj = MojaKlasa()
70 obj.atribut = 10
71 print(obj.atribut)
72 print(obj.atribut_binarny)
73 #Wynik: Ustawianie wartości atrybutu atrybut na 10
74 #       Pobieranie wartości atrybutu atrybut
75 #       10
76 #       0b1010
```

Klasyfikacja deskryptorów



Źródło:

Anaya Mariano, „Czysty kod w Pythonie. Twórz wydajny i łatwy w utrzymaniu kod”, Wydawnictwo Helion, Gliwice 2022

DEMO



Stwórz deskryptor, który będzie działał jako walidator email klasy **Student**. Klasa **Student** zawiera pola **imie**, **nazwisko** i **email**. Deskryptor ten powinien sprawdzać poprawność danych wprowadzanych podczas tworzenia lub modyfikowania instancji **Student**, czyli czy adres email jest poprawny.

```
1  # -*- coding: utf-8 -*-
2  import re
3  import logging
4
5  logging.basicConfig(level = logging.INFO)
6  class EmailValidator:
7      def __set__(self, instance, value):
8          if not re.match(r'^[a-zA-Z0-9._%+-]+@wsei.edu.pl$', value):
9              raise ValueError("Niepoprawny adres email. Adres musi być w domenie 'wsei.edu.pl'.")
10
11         instance.__dict__['email'] = value
12         logging.info(f"Zapisano poprawny adres email dla {instance.imie} {instance.nazwisko}: {value}")
13
14
15  class Student:
16      email = EmailValidator()
17
18      def __init__(self, imie, nazwisko, email):
19          self.imie = imie
20          self.nazwisko = nazwisko
21          self.email = email
22
23
24  student1 = Student("Jan", "Kowalski", "jan.kowalski@wsei.edu.pl")
25  print(student1.email)
26  # Wynik: jan.kowalski@wsei.edu.pl
27
28  student2 = Student("Anna", "Nowak", "anna.nowak@gmail.com")
29  # Wynik: ValueError
```

Zadanie



Stwórz klasę **Uzytkownik**. Klasa powinna zawierać atrybuty **imie** i **wiek**. Opracuj deskryptor, który będzie rejestrował dostęp do tych atrybutów za pomocą logowania. Deskryptor powinien logować informacje o **odczycie oraz zapisie** wartości atrybutu.

Generatory i iteratory



Generator w Pythonie to funkcja, która zwraca **iterator**, który produkuje sekwencję wartości, **zwracając je jedną po drugiej**, zamiast zwracać całą sekwencję od razu.

Iteratory **zwiększają wydajność** oraz **redukują zużycie pamięci**.

Są alternatywą dla innych obiektów iterowalnych lub kontenerów, które wymagają większej ilości miejsca w pamięci, takich jak **list**, **set** czy **tuple**.

Generatory są często stosowane do czytania dużych plików. Zamiast wczytywać cały plik do pamięci na raz, można użyć generatorów i iteracji, aby przetwarzać plik partiami lub liniami, co pozwala na minimalne zużycie pamięci

Generatory poprzez wyrażenia (*comprehension*)



Generator można zdefiniować (*list comprehension*) w podobny sposób jak wyrażenia listowe.

Jeśli w wyrażeniu listowym zastąpimy nawiasy kwadratowe nawiasami okrągłymi, otrzymamy wyrażenie generatorowe.

```
5 lista = [x**2 for x in range(10)]
6 print(type(lista))
7 #Wynik: <class 'list'>
8
9 generator = (x**2 for x in range(10))
10 print(type(generator))
11 #Wynik: <class 'generator'>
```

Dobra praktyka:

Zawsze przekazuj wyrażenie generatorowe, zamiast wyrażenia listowego, do funkcji, które oczekują obiektów iterowalnych, takich jak *min()*, *max()* i *sum()*.

```
13 #Uzycie w funkcjach wbudowanych
14 min(x**2 for x in range(10))
15 max(x**2 for x in range(10))
16 sum(x**2 for x in range(10))
```

yield



Instrukcja ***yield*** działa podobnie do *return*, ale zamiast zakończenia funkcji i zwrócenia wartości do miejsca wywołania, **tymczasowo zatrzymuje funkcję, zwracając wartość do miejsca wywołania**. Następnie, gdy funkcja zostanie wznowiona, będzie kontynuować działanie od miejsca, w którym została zatrzymana, zamiast rozpoczynać od nowa.

```
19 #Uzycie yield
20 def generator_liczb(n):
21     for i in range(n):
22         yield i
23
24 gen = generator_liczb(5)
25 for liczba in gen:
26     print(liczba)
27 #Wynik 0
28 #    1
29 #    2
30 #    3
31 #    4
```

next()



Funkcja **next()** jest używana do pobierania kolejnych wartości z generatora. Gdy generator jest wywoływany za pomocą funkcji next(), wykonuje on kolejny krok i zwraca kolejną wartość z sekwencji, którą generuje.

```
33 gen = generator_liczb(5)
34 print(next(gen)) # Wynik 0
35 print(next(gen)) # Wynik 1
36 print(next(gen)) # Wynik 2
37 print(next(gen)) # Wynik 3
38 print(next(gen)) # Wynik 4
39 print(next(gen, "wartosc domyslna"))
40 # Wynik wartosc domyslna
```

Gdy generator wyczerpie swoje wartości, a próbujemy pobrać kolejną za pomocą next(), zostanie wygenerowany wyjątek **StopIteration**. W pętli *for* jest on **automatycznie obsługiwany**, dlatego zazwyczaj używamy pętli *for* do iterowania po generatorach.

Możemy **dostarczyć** tej funkcji **wartość domyślną** w jej drugim parametrze.

itertools



Moduł **itertools** w Pythonie oferuje zestaw narzędzi do efektywnego przetwarzania i manipulacji iteracji, co może znacząco upraszczać kod, szczególnie w przypadku wielokrotnych iteracji.

Przykłady metod:

- **itertools.cycle()** - pozwala na cykliczne przeglądanie kolekcji.
- **itertools.repeat()** - powtarza określoną wartość określoną liczbę razy.
- **itertools.product()** - tworzy iloczyn kartezjański dwóch lub więcej zbiorów.
- **itertools.combinations()** - generuje kombinacje elementów z kolekcji bez powtórzeń.
- **itertools.permutations()** - generuje permutacje elementów z kolekcji.

Metody magiczne `__next__` i `__iter__`



Metody magiczne `__next__` i `__iter__` umożliwiają iterowanie przez obiekty klasy. Są elementami wykorzystywanymi przy tworzeniu klas, które zachowują się jak iteratory.

```
42 #Metody magiczne __next__ oraz __iter__
43 class MojIterator:
44     def __init__(self, data):
45         self.data = data
46         self.index = 0
47
48     def __iter__(self):
49         return self
50
51     def __next__(self):
52         if self.index >= len(self.data):
53             raise StopIteration
54         value = self.data[self.index]
55         self.index += 1
56         return value
57
58 moj_iterator = MojIterator([1, 2, 3])
59 for item in moj_iterator:
60     print(item)
61 #Wynik: 1
62 #      2
63 #      3
```

`__next__()`: ta metoda definiuje, co ma być zwrócone przy wywoływaniu funkcji **`next()`** na obiekcie klasy.

`__iter__()`: Ta metoda definiuje, co ma się dzieć, gdy obiekt klasy jest iterowany. Powinna zwracać iterator, który będzie używany do iteracji przez obiekt.

close()



W momencie wywoływania metody `close()` generator kończy swoje działanie, a jego iteracja zostanie zatrzymana.

```
66 #Uzycie .close()
67 generator = (x**2 for x in range(10))
68 for i in generator:
69     if i ==9:
70         generator.close()
71         print(i)
72 #Wynik: 0
73 #      1
74 #      4
75 #      9
```

throw(ex_type[, ex_value[, ex_traceback]])



Metoda ***throw()*** jest używana w kontekście obsługi wyjątków w generatorach. Gdy metoda *throw()* zostanie wywołana, **wyjątek zostanie wywołany w miejscu, gdzie generator jest obecnie zatrzymany**, a następnie generator będzie mógł go obsłużyć za pomocą konstrukcji *try-except*.

```
78 #Uzycie metody throw
79 def counter():
80     try:
81         count = 0
82         while True:
83             yield count
84             count += 1
85     except StopIteration:
86         print("Generator został zatrzymany.")
87
88 cnt = counter()
89 for i in cnt:
90     print(i)
91     if i >= 2:
92         cnt.throw(StopIteration)
93 #Wynik: 0
94 #      1
95 #      2
96 #      StopIteration
```


send(value)



Metoda ***send()*** jest używana w kontekście komunikacji dwukierunkowej między kodem wywołującym a generatorem. Pozwala ona na przekazanie wartości do generatora w dowolnym momencie jego działania i umożliwia generatorowi na przetworzenie tej wartości i ewentualne zwrócenie wyniku.

```
96     #Uzycie metody send()
97     def dodawanie():
98         result = 0
99         while True:
100             value = yield result
101             result += value
102
103
104     calc = dodawanie()
105     next(calc)
106
107     print(calc.send(5))
108     #Wynik 5
109     print(calc.send(3))
110     #Wynik 8
111     print(calc.send(10))
112     #Wynik 18
```

DEMO



Napisz która będzie implementowała generatorklasę, Fibonacciego za pomocą metod magicznych ciągu `__iter__()` i `__next__()`.

```
1  #Napisz klasę, która będzie implementowała generator ciągu Fibonacciego
2  class FibonacciGenerator:
3      def __init__(self, limit):
4          self.prev = 0
5          self.curr = 1
6          self.limit = limit
7
8      def __iter__(self):
9          return self
10
11     def __next__(self):
12         if self.limit >= self.prev:
13             fib = self.prev
14             self.prev = self.curr
15             self.curr = self.prev + self.curr
16             return fib
17         else:
18             raise StopIteration
19
20     fib_gen = FibonacciGenerator(132)
21     for fib in fib_gen:
22         print(fib)
23
```

zadanie



Opracuj generator ciągu Collatza Dla liczby naturalnej n , jeśli n jest parzyste, dziel przez 2; jeśli n jest nieparzyste, pomnóż przez 3 i dodaj 1,

$$a_{n+1} = \begin{cases} a_n/2 & \text{dla } a_n \text{ parzystych} \\ 3 * a_n + 1 & \text{dla } a_n \text{ nieparzystych} \end{cases}$$

zaczynając od określonej liczby początkowej, aż do osiągnięcia wartości 1.



● REKRUTACJA OTWARTA

AI KRAK HACK

27 - 28.03.2026

Hackathon WSEI



Zmierzymy się po raz kolejny z wyzwaniami naszego miejskiego otoczenia, wykorzystując potencjał optymalizacji, uczenia maszynowego i sztucznej inteligencji!

ZGŁOŚ SIĘ TERAZ

HARMONOGRAM →

WYZWANIA AI KRAK HACK 2026

 krakhack.info



Dwa główne wyzwania czekają na uczestników. Wybierz swój track i stwórz rozwiązanie, które zmieni przyszłość krakowskiej infrastruktury.



ACTIVE TRACK

SMART INFRASTRUCTURE CHALLENGE

INFRASTRUKTURA PRZYSZŁOŚCI OPARTA NA DANYCH

Zaprojektuj inteligentną infrastrukturę lokalną wykorzystując analizę danych geoprzestrzennych, optymalizację tras i modelowanie przestrzenne.

GIS

PYTHON

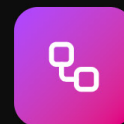
POSTGIS

MACHINE LEARNING

OPTIMIZATION

GEOSPATIAL ANALYSIS

[ZOBACZ WIĘCEJ DETALI ZADANIA](#) →



ACTIVE TRACK

PROCESS-TO-AUTOMATION COPILOT

OD DANYCH PROCESOWYCH DO AUTOMATYZACJI

Zbuduj inteligentnego asystenta, który przekształca dane o procesach biznesowych w gotową automatyzację workflow. Wykorzystaj process mining, AI agents i workflow engines.

PROCESS MINING

BPMN

AI AGENTS

WORKFLOW ENGINES

PYTHON

CAMUNDA

[ZOBACZ WIĘCEJ DETALI ZADANIA](#) →